1. Python Programs related to variables and expression showing impact of operators.

1) Arithmetic Operators

```
a = 10

b = 3

print("Addition:", a + b)

print("Subtraction:", a - b)

print("Multiplication:", a * b)

print("Division:", a / b)

print("Modulus:", a % b)
```

#OUTPUT:-

```
Addition: 13
Subtraction: 7
Multiplication: 30
Division: 3.3333333333333335
Modulus: 1
```

2) Assignment Operators

```
x = 5

x += 3   # x = x + 3

print("After += :", x)

x *= 2   # x = x * 2

print("After *= :", x)

x -= 4   # x = x - 4

print("After -= :", x)
```

#OUTPUT:-

```
After += : 8
After *= : 16
After -= : 12
```

3) Logical Operators

```
x = True

y = False

print("AND:", x and y)

print("OR:", x or y)

print("NOT x:", not x)
```

#OUTPUT:-

```
AND: False
OR: True
NOT x: False
```

4) Expression Evaluation Example

```
a = 5

b = 2

c = 3

result = a + b * c

print("Result:", result)
```

#OUTPUT:-

```
Result: 11
```

2. Programs related to the conditional statements which shows different impact of If() and ladder if().

1) Using Multiple `if`

```
num = 10

if num > 0:

   print("Positive")

if num % 2 == 0:

   print("Even")
```

#OUTPUT:-

```
Positive
Even
```

2) Using if-elif-else Ladder

```
num = 10

if num > 0:

   print("Positive")

elif num % 2 == 0:

   print("Even")
```

#OUTPUT:-

```
Positive
```

3. Programs related to the iterative statements which shows different impact of for() and nested for() and while() loop including break and continue statements.

1) for loop()

```
# Print numbers from 1 to 5

for i in range(1, 6):

    print(i)
```

#OUTPUT:-

```
1
2
3
4
5
```

2)nested for loop()

```
# Print pattern

for i in range(1, 4):

    for j in range(1, 4):

        print("*", end=" ")

    print()
```

#OUTPUT:-

```
* * *
* * *
* * *
```

3)while loop()

```
# Print numbers from 1 to 5

i = 1

while i <= 5:

    print(i)

    i += 1
```

#OUTPUT:-

```
1
2
3
4
5
```

4) Break statement

```
# Stop loop when number is 3
```

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

#OUTPUT:-

```
1
2
```

5) Continue statement

```
# Skip number 3
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

#OUTPUT:-

```
1
2
4
5
```

4. Programs related to the string manipulation using conditional statements and iterations.

1) Check Vowels in a String

```
string = input("Enter a string: ")

count = 0

for ch in string:

    if ch.lower() in 'aeiou':

        count += 1

print("Number of vowels:", count)
```

#OUTPUT:-

```
Enter a string: Hello Neha
Number of vowels: 4
```

2) Count digits, letters, spaces

```
# Count different characters

string = input("Enter a string: ")

letters = digits = spaces = 0

for ch in string:

    if ch.isalpha():

        letters += 1

    elif ch.isdigit():

        digits += 1

    elif ch.isspace():

        spaces += 1

print("Letters:", letters)

print("Digits:", digits)

print("Spaces:", spaces)
```

#OUTPUT:-

```
Enter a string: Hello 12345 World
Letters: 10
Digits: 5
Spaces: 2
```

5. Programs related to the string built in functions and their uses in various manners.

1) upper() and lower()

```
# Convert string case

text = input("Enter a string: ")

print("Uppercase:", text.upper())

print("Lowercase:", text.lower())
```

#OUTPUT:-

```
Enter a string: Hello Neha
Uppercase: HELLO NEHA
Lowercase: hello neha
```

2) strip()

```
# Remove spaces

text = input("Enter a string with spaces: ")

print("After strip:", text.strip())
```

#OUTPUT:-

```
Enter a string with spaces:  Hello Neha
After strip: Hello Neha
```

3) replace()

```
# Replace word

text = input("Enter a string: ")

new_text = text.replace("Python", "Java")

print("Updated string:", new_text)
```

#OUTPUT:-

```
Enter a string: Python is easy and Python is powerful.
Updated string: Java is easy and Java is powerful.
```

4) find()

```
# Find position of substring

text = input("Enter a string: ")

pos = text.find("a")

print("Position of 'a':", pos)
```

#OUTPUT:-

```
Enter a string: Python Programming
Position of 'a': 12
```

5) count()

```
# Count occurrences

text = input("Enter a string: ")

print("Count of 'a':", text.count("a"))
```

#OUTPUT:-

```
Enter a string: Hello Neha Kiran Ghegade
Count of 'a': 3
```

6) split()

```
# Split string into list

text = input("Enter a sentence: ")

words = text.split()

print("Words:", words)
```

#OUTPUT:-

```
Enter a sentence: Python is easy and powerful programming language
Words: ['Python', 'is', 'easy', 'and', 'powerful', 'programming', 'language']
```

7) len()

```
# Find length

text = input("Enter a string: ")

print("Length:", len(text))
```

#OUTPUT:-

```
Enter a string: Hello Neha
Length: 10
```

6. Programs related to the function which implies the different effect of passing parameters and arguments.

1) positional vs keywords arguments

```
def student(name, age):

    print("Name:", name)

    print("Age:", age)

# Positional

student("Neha", 20)

# Keyword

student(age=20, name="Neha")
```

#OUTPUT:-

```
Name: Neha
Age: 20
Name: Neha
Age: 20
```

2) Immutable vs Mutable Argument

```
# Immutable (int)

def change(x):

    x = x + 5

    print("Inside:", x)

num = 10

change(num)

print("Outside:", num)

# Mutable (list)

def modify(lst):

    lst.append(100)

numbers = [1, 2, 3]

modify(numbers)

print("List:", numbers)
```

#OUTPUT:-

```
Inside: 15
Outside: 10
List: [1, 2, 3, 100]
```

7. Programs related to the List, tuple, Set, and Dictionary which shows the impact on real implementation.

1) List

```
marks = [80, 75, 90, 60]

# Add new marks

marks.append(85)

# Update marks

marks[1] = 78

print("Student Marks:", marks)
```

#OUTPUT:-

```
Student Marks: [80, 78, 90, 60, 85]
```

2) Tuple

```
days = ("Mon", "Tue", "Wed", "Thu", "Fri")

print("Days:", days)
```

#OUTPUT:-

```
Days: ('Mon', 'Tue', 'Wed', 'Thu', 'Fri')
```

3) Set

```
numbers = {1, 2, 3, 2, 1}

print("Unique numbers:", numbers)

numbers.add(5)

print("After adding:", numbers)
```

#OUTPUT:-

```
Unique numbers: {1, 2, 3}
After adding: {1, 2, 3, 5}
```

4) Dictionary

```
student = {

   "name": "Neha",

   "age": 20,

   "marks": 85

}

# Access data
```

```
print("Name:", student["name"])

# Update data

student["marks"] = 90

print("Updated:", student)
```

#OUTPUT:-

```
Name: Neha
Updated: {'name': 'Neha', 'age': 20, 'marks': 90}
```

8. Programs related to Data Cleaning and Preparation.

1) Remove extra spaces

```
text = "   Data Science   "

clean_text = text.strip()

print("Before:", text)

print("After:", clean_text)
```

#OUTPUT:-

```
Before:    Data Science
After: Data Science
```

2) Handle missing values

```
# Replace missing values

data = ["Neha", "", "Snehal", None, "Pallavi"]

clean_data = []

for item in data:

    if item == "" or item is None:

        clean_data.append("Unknown")

    else:

        clean_data.append(item)

print("Clean Data:", clean_data)
```

#OUTPUT:-

```
Clean Data: ['Neha', 'Unknown', 'Snehal', 'Unknown', 'Pallavi']
```

3) Convert Data type

```
# Convert string to integer

data = ["10", "20", "30", "40"]

numbers = []

for item in data:

    numbers.append(int(item))

print("Converted Data:", numbers)
```

#OUTPUT:-

```
Converted Data: [10, 20, 30, 40]
```

9. Programs to use the Panda and related functions to show the data analysis and their relations.

1) Basic data analysis

```
import pandas as pd

data = {

   "Name": ["Neha", "Snehal", "Pallavi", "Sneha"],

   "Marks": [85, 90, 75, 80]

}

df = pd.DataFrame(data)

print("Data:\n", df)

print("Average Marks:", df["Marks"].mean())

print("Maximum Marks:", df["Marks"].max())
```

#OUTPUT:-

```
Data:
       Name  Marks
0     Neha     85
1   Snehal     90
2  Pallavi     75
3    Sneha     80
Average Marks: 82.5
Maximum Marks: 90
```

2) Finding Relationship (Correlation)

```
import pandas as pd

data = {

   "Study_Hours": [2, 4, 6, 8],

   "Marks": [50, 60, 70, 90]

}

df = pd.DataFrame(data)

# Correlation

correlation = df["Study_Hours"].corr(df["Marks"])

print("Correlation between Study Hours and Marks:", correlation)
```

#OUTPUT:-

```
Correlation between Study Hours and Marks: 0.982707629823991
```

10. Programs related to implementation of Data Wrangling: Join, Combine, and Reshape

1) Join (Merge)

```
import pandas as pd

df1 = pd.DataFrame({

    "ID": [1, 2, 3],

    "Name": ["Neha", "Rahul", "Amit"]

})

df2 = pd.DataFrame({

    "ID": [1, 2, 3],

    "Marks": [85, 90, 75]

})

# Merge

result = pd.merge(df1, df2, on="ID")

print(result)
```

#OUTPUT:-

```
   ID   Name  Marks
0   1   Neha     85
1   2  Rahul     90
2   3   Amit     75
```

2) Reshape (Pivot)

```
import pandas as pd

data = {

    "Name": ["Neha", "Neha", "Rahul", "Rahul"],

    "Subject": ["Math", "Sci", "Math", "Sci"],

    "Marks": [80, 85, 70, 75]

}

df = pd.DataFrame(data)

pivot = df.pivot(index="Name", columns="Subject", values="Marks")
```

print(pivot)

#OUTPUT:-

```
Subject  Math  Sci
Name
Neha       80   85
Rahul      70   75
```

11. Programs related to implementation Categorical Data & Advanced GroupBy.

1) GroupBy

```
import pandas as pd

data = {

   "Dept": ["IT", "IT", "HR", "HR"],

   "Salary": [50000, 60000, 45000, 40000]

}

df = pd.DataFrame(data)

group = df.groupby("Dept")["Salary"].mean()

print(group)
```

#OUTPUT:-

```
Dept
HR    42500.0
IT    55000.0
Name: Salary, dtype: float64
```

2) Categorical Data

```
import pandas as pd

data = ["Low", "High", "Medium", "Low"]

cat = pd.Categorical(data, categories=["Low", "Medium", "High"], ordered=True)

print(cat)
```

#OUTPUT:-

```
['Low', 'High', 'Medium', 'Low']
Categories (3, object): ['Low' < 'Medium' < 'High']
```

12. Programs related to implementation Advanced Pandas Techniques.

1) Apply Function

```
import pandas as pd

data = {

   "Marks": [50, 60, 70]

}

df = pd.DataFrame(data)

# Increase marks by 5

df["Updated"] = df["Marks"].apply(lambda x: x + 5)

print(df)
```

#OUTPUT:-

```
   Marks  Updated
0     50       55
1     60       65
2     70       75
```

2) Sorting

```
import pandas as pd

data = {

   "Name": ["Neha", "Rahul", "Amit"],

   "Marks": [85, 90, 75]

}

df = pd.DataFrame(data)

sorted_df = df.sort_values(by="Marks", ascending=False)

print(sorted_df)
```

#OUTPUT:-

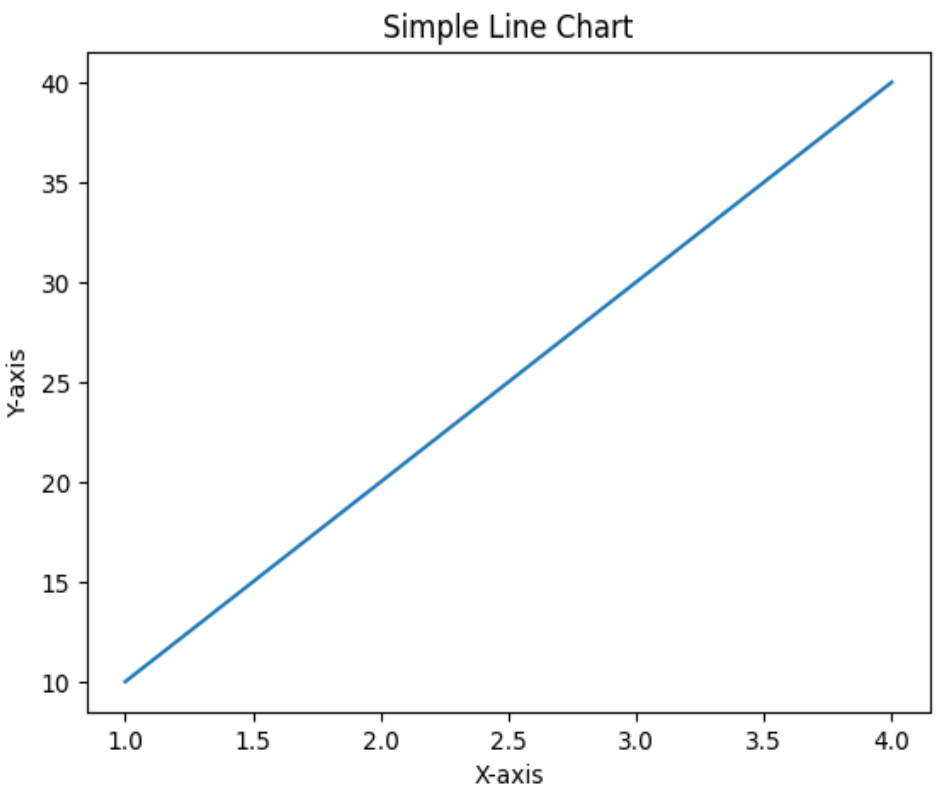
```
     Name  Marks
1   Rahul     90
0    Neha     85
2    Amit     75
```

13. Programs related Programs using NumPy and least usable functions of NumPy like modification of chart, plot, and labels. to file handling to show the open, read, write, and append execution.

1) NumPy + Plot

```
import numpy as np

import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4])

y = np.array([10, 20, 30, 40])

plt.plot(x, y)

plt.title("Simple Line Chart")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.show()
```

#OUTPUT:-



2) File Handling

```
# Write to file

file = open("data.txt", "w")

file.write("Hello Neha\n")

file.close()

# Read file

file = open("data.txt", "r")

print(file.read())

file.close()

# Append file

file = open("data.txt", "a")

file.write("Welcome!\n")

file.close()
```

#OUTPUT:-

```
Hello Neha
```

14. Programs related to indexing, slicing, sorting, and matrix manipulation.

1) Indexing and Slicing

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])

print("First element:", arr[0])

print("Slice (1 to 3):", arr[1:4])
```

#OUTPUT:-

```
First element: 10
Slice (1 to 3): [20 30 40]
```

2) Sorting

```
import numpy as np

arr = np.array([50, 10, 40, 20])

sorted_arr = np.sort(arr)

print("Sorted array:", sorted_arr)
```

#OUTPUT:-

```
Sorted array: [10 20 40 50]
```

3) Matrix Manipulation

```
import numpy as np

A = np.array([[1, 2], [3, 4]])

B = np.array([[5, 6], [7, 8]])

print("Addition:\n", A + B)

print("Multiplication:\n", A.dot(B))
```

#OUTPUT:-

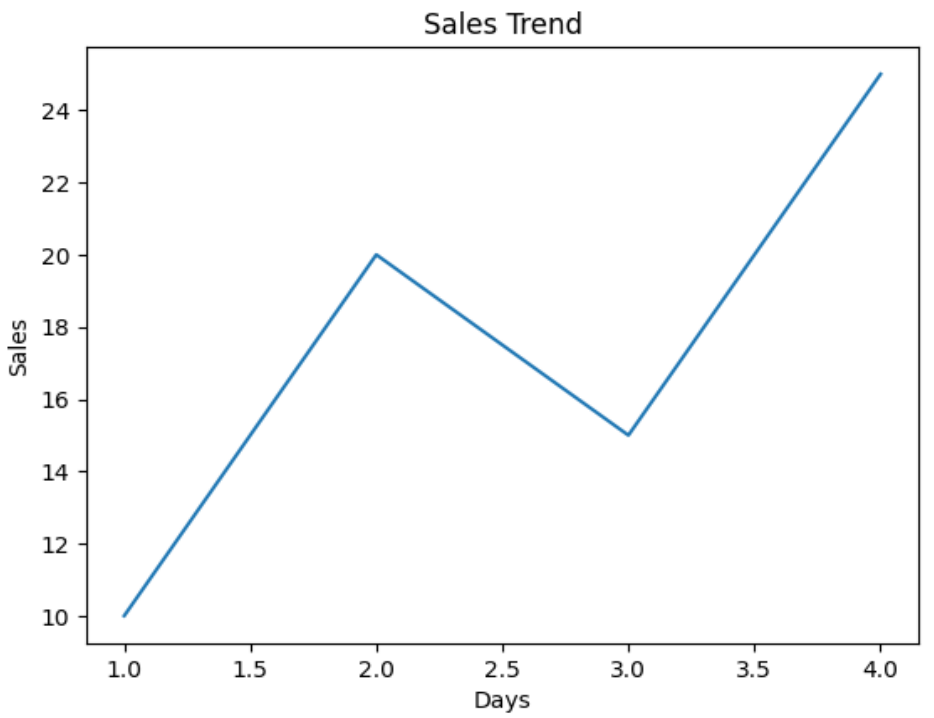
```
Addition:
 [[ 6  8]
 [10 12]]
Multiplication:
 [[19 22]
 [43 50]]
```

15. Programs using NumPy and least usable functions of NumPy like modification of chart, plot, and labels.

Chart with Labels

```
import numpy as np

import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4])

y = np.array([10, 20, 15, 25])

plt.plot(x, y)

plt.title("Sales Trend")

plt.xlabel("Days")

plt.ylabel("Sales")

plt.show()
```

#OUTPUT:-



16. Programs using NumPy to show the classification and clustering of data accessing through storage file.

1) Basic Classification

```
import numpy as np

marks = np.array([35, 60, 80, 45, 90])

# Classification

result = []

for m in marks:

    if m >= 50:

        result.append("Pass")

    else:

        result.append("Fail")

print(result)
```

#OUTPUT:-

```
['Fail', 'Pass', 'Pass', 'Fail', 'Pass']
```

2) Simple Clustering

```
import numpy as np

data = np.array([10, 12, 50, 55, 11, 52])

low = []

high = []

for d in data:

    if d < 30:

        low.append(d)

    else:

        high.append(d)

print("Cluster 1:", low)

print("Cluster 2:", high)
```

#OUTPUT:-

```
Cluster 1: [np.int64(10), np.int64(12), np.int64(11)]
Cluster 2: [np.int64(50), np.int64(55), np.int64(52)]
```

17. Programs related to implementation of Array and Data Analytics.

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])

print("Mean:", np.mean(data))

print("Max:", np.max(data))

print("Min:", np.min(data))
```

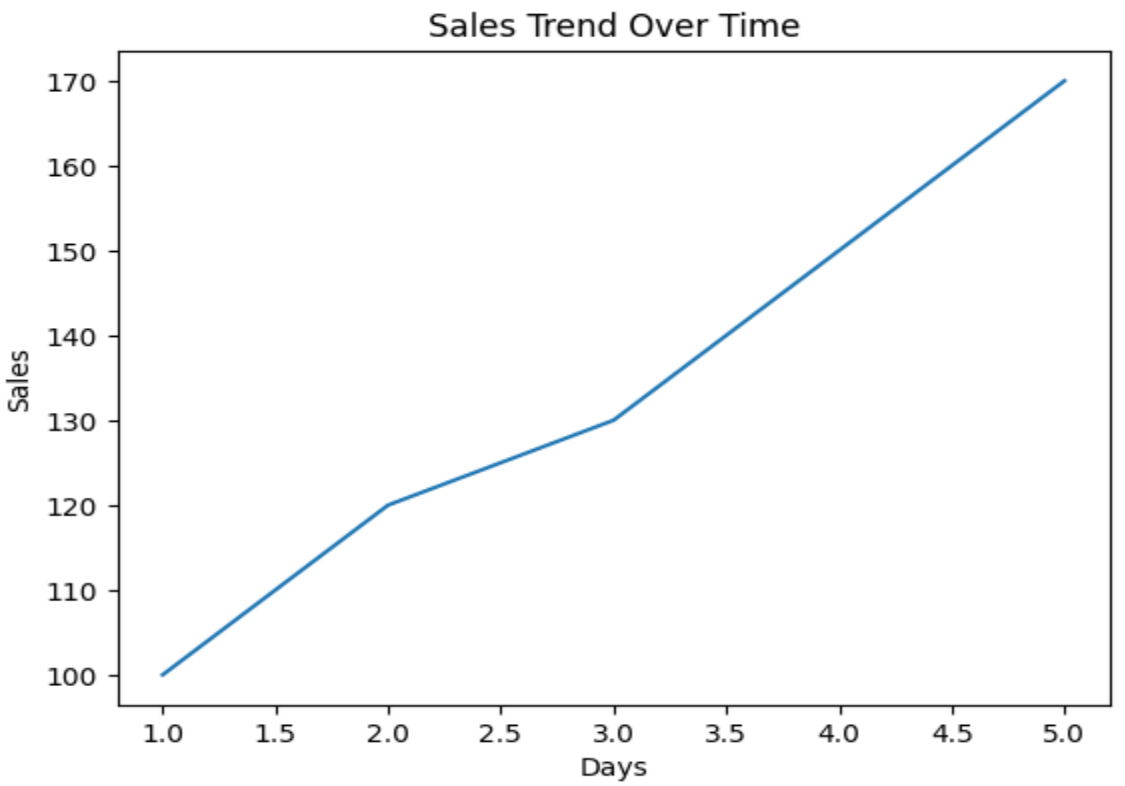
#OUTPUT:-

```
Mean: 30.0
Max: 50
Min: 10
```

18. Program to implement Time-Series & Trend Analysis.

```
import numpy as np

import matplotlib.pyplot as plt

days = np.array([1, 2, 3, 4, 5])

sales = np.array([100, 120, 130, 150, 170])

plt.plot(days, sales)

plt.title("Sales Trend Over Time")

plt.xlabel("Days")

plt.ylabel("Sales")

plt.show()
```

#OUTPUT:-



19. Program to implement Statistical & Correlation Analysis.

```
import pandas as pd

data = {

    "Study_Hours": [2, 4, 6, 8],

    "Marks": [50, 60, 70, 90]

}

df = pd.DataFrame(data)



# Statistical analysis

print("Mean Marks:", df["Marks"].mean())

print("Max Marks:", df["Marks"].max())



# Correlation

corr = df["Study_Hours"].corr(df["Marks"])

print("Correlation:", corr)
```

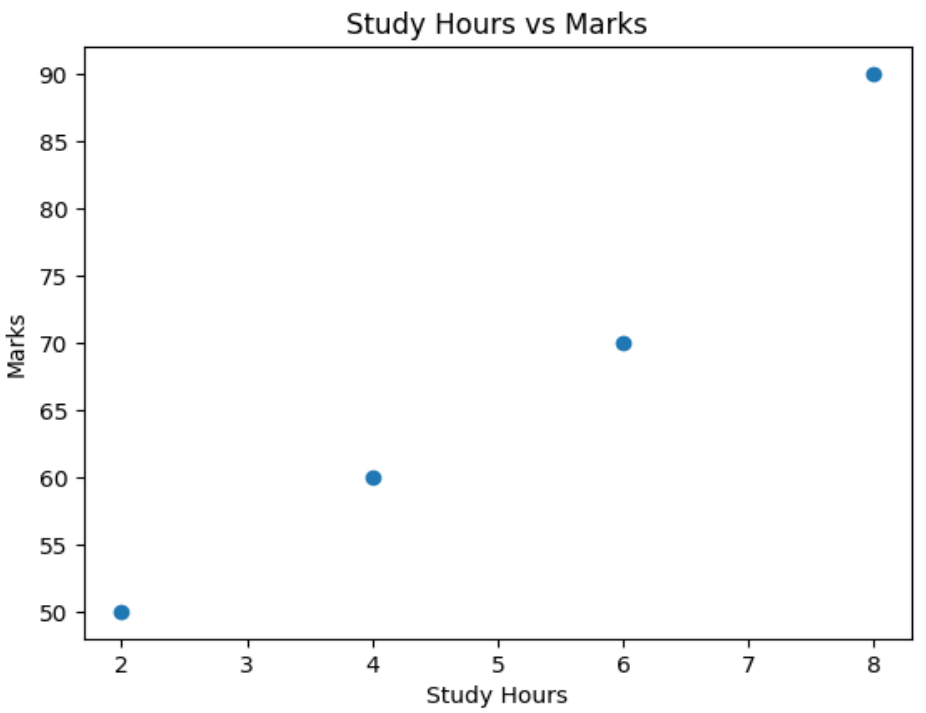
#OUTPUT:-

```
Mean Marks: 67.5
Max Marks: 90
Correlation: 0.982707629823991
```

20. Program to implement Visualization for Relationship Analysis (Pandas + Matplotlib/Seaborn).

```
import pandas as pd

import matplotlib.pyplot as plt

data = {

    "Study_Hours": [2, 4, 6, 8],

    "Marks": [50, 60, 70, 90]

}

df = pd.DataFrame(data)

# Scatter plot

plt.scatter(df["Study_Hours"], df["Marks"])

plt.title("Study Hours vs Marks")

plt.xlabel("Study Hours")

plt.ylabel("Marks")

plt.show()
```

#OUTPUT:-



21. Program to implement Dependence and Independence, Conditional Probability.

```
# Probability example

# P(A) = Probability of passing

# P(B) = Probability of attending class

P_A = 0.6

P_B = 0.5

P_A_and_B = 0.3


# Check independence

if P_A * P_B == P_A_and_B:

    print("Independent Events")

else:

    print("Dependent Events")


# Conditional Probability

P_A_given_B = P_A_and_B / P_B


print("P(A|B):", P_A_given_B)
```

#OUTPUT:-

```
Independent Events
P(A|B): 0.6
```

22. Program to implement Bayes's Theorem.

```
# Bayes Theorem

P_A = 0.6       # Prior probability

P_B_given_A = 0.7

P_B = 0.5


# Bayes formula

P_A_given_B = (P_B_given_A * P_A) / P_B


print("P(A|B):", P_A_given_B)
```

#OUTPUT:-

```
P(A|B): 0.84
```

23. Program to implement Statistical Hypothesis: Flipping a Coin

```
import numpy as np


# Simulate 100 coin tosses (0 = Tail, 1 = Head)

tosses = np.random.choice([0, 1], size=100)

heads = np.sum(tosses)

tails = 100 - heads

print("Heads:", heads)

print("Tails:", tails)


# Hypothesis: coin is fair (approx equal heads & tails)

if abs(heads - tails) < 20:

    print("Accept Null Hypothesis (Coin is Fair)")

else:

    print("Reject Null Hypothesis (Coin is Biased)")
```

#OUTPUT:-

```
Heads: 64
Tails: 36
Reject Null Hypothesis (Coin is Biased)
```

24. Program to implement p-Values and p-Hacking

```
import numpy as np

from scipy import stats


# Sample data (e.g., marks of students)

data = [50, 55, 60, 65, 70, 75, 80]


# Test mean = 60

t_stat, p_value = stats.ttest_1samp(data, 60)

print("p-value:", p_value)


# Decision

if p_value < 0.05:

    print("Reject Null Hypothesis")

else:

    print("Accept Null Hypothesis")
```

#OUTPUT:-

```
p-value: 0.266569703380069
Accept Null Hypothesis
```